Inventory Sales Management System

Course: Database Techniques (DV1703)

Student:
Afif Al Mohammed

Blekinge Institute of Technology (BTH)

Date: March 2026

1. Project Idea

In this project, we designed and implemented an Inventory and Sales Management System. The goal of the system is to help manage products, customers, and sales in a small store.

The system allows the user to store information about products, suppliers, and product categories. It also stores customer information and sales transactions.

Using the system, the user can view products, create new sales, and analyse sales data. The system can also calculate the total sales per transaction, the total amount spent by each customer, and the most purchased product category for each gender.

One important feature of the system is inventory control. The database prevents selling products when there is not enough stock available. This is implemented using database triggers.

The data used in the system was generated manually to simulate a small retail environment.

2. Schema Design

ER Diagram

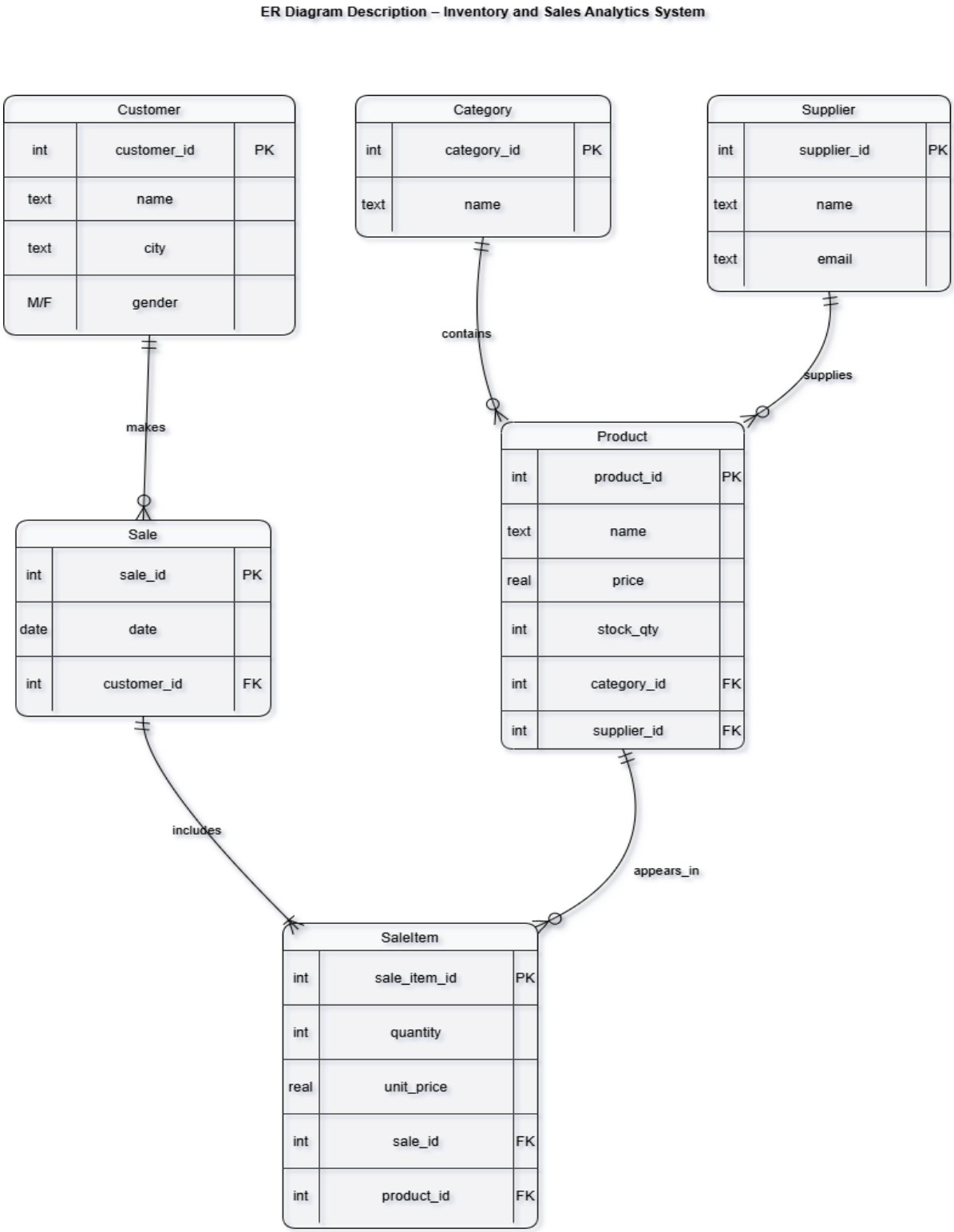

FIGURE 1 *shows the ER diagram of the database schema*

The database schema consists of six main tables: Customer, Category, Supplier, Product, Sale, and SaleItem.

The Customer table stores information about customers such as their name, city, and gender.

The Category table stores product categories. Each product belongs to one category.

The Supplier table stores information about suppliers who provide products.

The Product table stores product information including the product name, price, stock quantity, and references to its category and supplier.

The Sale table represents a sales transaction and stores the date of the sale and the customer who made the purchase.

The SaleItem table stores the individual items in each sale. It connects products and sales and stores the quantity and price of each product sold.

Relationships between the tables are implemented using foreign keys. For example, each sale references a customer, and each sale item references both a sale and a product.

3. SQL Queries

Several SQL queries were implemented to support the functionality of the system.

Query 1 – Product Overview

Q: Show all products together with their category and supplier information.

This query joins the Product, Category, and Supplier tables in order to display complete information about each product. The JOIN operation is used to connect the tables using their foreign key relationships.

```
SELECT
    Product.product_id,
    Product.name AS product_name,
    Product.price,
    Product.stock_qty,
    Category.name AS category_name,
    Supplier.name AS supplier_name
FROM Product
JOIN Category ON Product.category_id = Category.category_id
JOIN Supplier ON Product.supplier_id = Supplier.supplier_id;
```

Query 2 – Total Amount per Sale

Q: Calculate the total value of each sale.

This query calculates the total value of each sale by multiplying the quantity of each sold product with its unit price. The SUM aggregation function is used together with GROUP BY to compute the total amount for each sale.

```
SELECT

    Sale.sale_id,
    Sale.sale_date,
    SUM(SaleItem.quantity * SaleItem.unit_price) AS total_amount

FROM Sale
JOIN SaleItem ON Sale.sale_id = SaleItem.sale_id
GROUP BY

Sale.sale_id,

Sale.sale_date;
```

Query 3 – Total Spending per Customer

Q: Show how much money each customer has spent in total.

This query joins the Customer, Sale, and SaleItem tables. The SUM function is used to calculate the total amount spent by each customer across all sales.

```
SELECT

    Customer.customer_id,
    Customer.name AS customer_name,
    SUM(SaleItem.quantity * SaleItem.unit_price) AS total_spent

FROM Customer
JOIN Sale ON Customer.customer_id = Sale.customer_id
JOIN SaleItem ON Sale.sale_id = SaleItem.sale_id
GROUP BY

Customer.customer_id,

Customer.name;
```

Query 4 – Total Spending by Gender and Category

Q: Analyze how much each gender spends on different product categories.

This query joins several tables including Customer, Sale, SaleItem, Product, and Category. The result is grouped by gender and category in order to analyse purchasing behavior.

```
SELECT
```

```
        Customer.gender,
        Category.name AS category_name,
        SUM(SaleItem.quantity * SaleItem.unit_price) AS total_spent

FROM Customer
JOIN Sale ON Customer.customer_id = Sale.customer_id
JOIN SaleItem ON Sale.sale_id = SaleItem.sale_id
JOIN Product ON SaleItem.product_id = Product.product_id
JOIN Category ON Product.category_id = Category.category_id
GROUP BY

Customer.gender,

 Category.name;
```

Query 5 – Most Purchased Category per Gender

Q: Find the product category with the highest total spending for each gender.

This query is based on a previous query (Query 4) that calculates total spending by gender and category.

This query uses a correlated subquery. First, total spending per gender and category is calculated. Then the query selects the category with the maximum spending for each gender.

```
SELECT

        gender_category_totals.gender,
        gender_category_totals.category_name,
        gender_category_totals.total_spent

FROM gender_category_totals
WHERE gender_category_totals.total_spent =

(

        SELECT

                MAX(gender_category_totals_inner.total_spent)
                FROM gender_category_totals AS gender_category_totals_inner
                WHERE gender_category_totals_inner.gender =
                gender_category_totals.gender

);
```

4. Discussion and Resources

During this project we designed and implemented a relational database system for managing products, customers, and sales in a small store environment.

The database was designed using six main tables: Customer, Category, Supplier, Product, Sale, and SaleItem. These tables are connected using foreign key relationships to maintain data integrity.

Several SQL queries were implemented to analyses the stored data. Some queries join multiple tables, while others use aggregation functions such as SUM and GROUP BY to calculate totals.

One of the queries is built on top of another query. For example, the query that finds the most purchased category per gender uses the results of a previous query that calculates total spending by gender and category. This demonstrates how complex queries can be built using intermediate results.

The project also uses database triggers. One trigger checks the product stock before inserting a sale item to prevent selling products when the stock is insufficient. Another trigger automatically reduces the stock quantity after a sale is successfully inserted.

A stored procedure was implemented to simplify the creation of a new sale. This procedure inserts a new sale and automatically creates the corresponding sale item. The triggers then ensure that stock validation and updates are handled automatically.

The project was implemented using MySQL and Python.

The database includes tables, views, triggers and a stored procedure.

The Python application connects to the database using the mysql-connector-python library and displays results using the tabulate library.

Source code:

https://github.com/username/InventorySalesProject

Database dump:

inventory_sales_dump.sql